

PostgreSQL SSL/TLS Configuration

Table of Contents

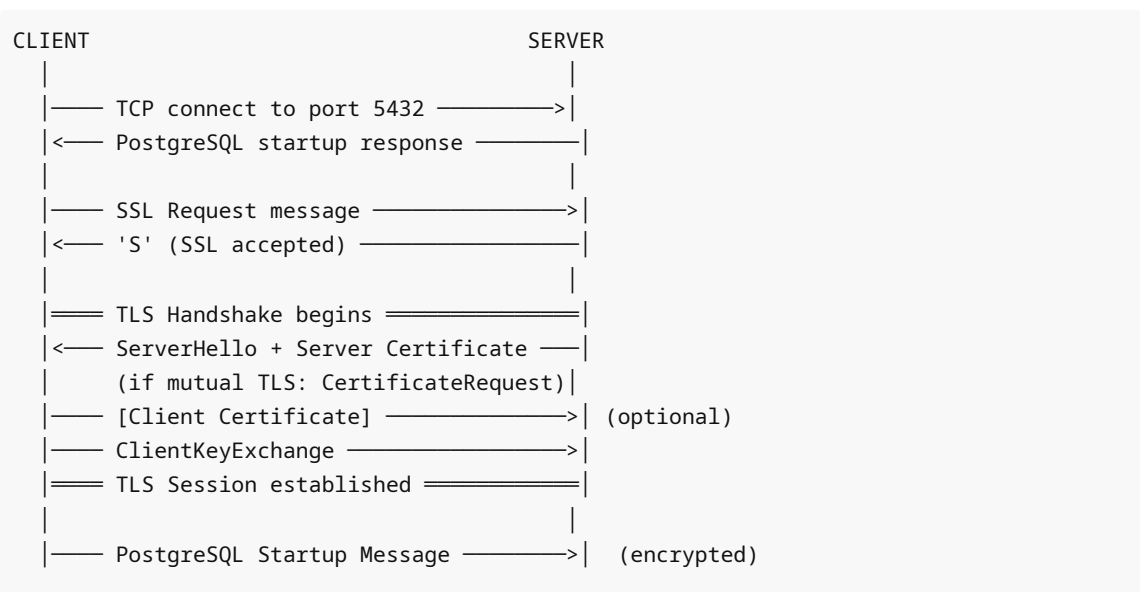
- 1. [Learning Objectives](#)
- 2. [SSL/TLS Architecture](#)
- 3. [Server Certificate Setup](#)
- 4. [Client Certificate Authentication](#)
- 5. [SSL Mode Options](#)
- 6. [Certificate Verification](#)
- 7. [Mutual TLS \(mTLS\)](#)
- 8. [Certificate Rotation](#)
- 9. [TLS Configuration Hardening](#)
- 10. [Monitoring SSL Connections](#)
- 11. [Common Mistakes](#)
- 12. [Best Practices](#)
- 13. [Performance Considerations](#)
- 14. [Interview Questions](#)
- 15. [Exercises and Solutions](#)

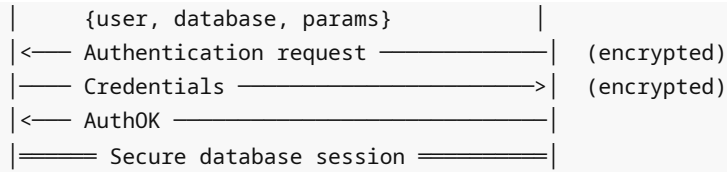
Learning Objectives

By the end of this module, you will be able to:

- Set up SSL/TLS on a PostgreSQL server from scratch
- Configure client certificate authentication
- Choose appropriate `sslmode` settings for different scenarios
- Implement certificate rotation without downtime
- Harden TLS configuration (cipher suites, protocol versions)
- Monitor and audit SSL connection usage

SSL/TLS Architecture





Server Certificate Setup

Using Self-Signed Certificates (Development/Testing)

```
# Generate self-signed certificate for the PostgreSQL server

# Step 1: Generate CA key and certificate
openssl genrsa -out ca.key 4096
openssl req -new -x509 -days 3650 -key ca.key -out ca.crt \
    -subj "/C=US/ST=California/O=MyCompany/CN=PostgreSQL CA"

# Step 2: Generate server private key
openssl genrsa -out server.key 2048

# Step 3: Create certificate signing request (CSR)
openssl req -new -key server.key -out server.csr \
    -subj "/C=US/ST=California/O=MyCompany/CN=db.example.com"

# Add Subject Alternative Names (required for modern clients)
cat > san.ext << EOF
[req]
req_extensions = v3_req
[v3_req]
subjectAltName = @alt_names
[alt_names]
DNS.1 = db.example.com
DNS.2 = postgres.internal
IP.1 = 192.168.1.100
IP.2 = 127.0.0.1
EOF

openssl req -new -key server.key -out server.csr \
    -config san.ext

# Step 4: Sign the CSR with CA
openssl x509 -req -days 365 -in server.csr -CA ca.crt -CAkey ca.key \
    -CAcreateserial -out server.crt -extensions v3_req -extfile san.ext

# Step 5: Set permissions
chmod 600 server.key ca.key
chmod 644 server.crt ca.crt
chown postgres:postgres server.key server.crt ca.crt

# Step 6: Place in PostgreSQL data directory
```

```
cp server.key server.crt ca.crt /etc/postgresql/16/main/  
# OR specify full paths in postgresql.conf
```

Using Let's Encrypt (Production with Public Hostname)

```
# Install certbot  
sudo apt-get install -y certbot  
  
# Get certificate for database hostname  
sudo certbot certonly --standalone -d db.example.com  
  
# Certificates location:  
# /etc/letsencrypt/live/db.example.com/fullchain.pem (server cert + chain)  
# /etc/letsencrypt/live/db.example.com/privkey.pem   (private key)  
# /etc/letsencrypt/live/db.example.com/chain.pem     (CA chain only)  
  
# Link to PostgreSQL-accessible location  
sudo ln -sf /etc/letsencrypt/live/db.example.com/fullchain.pem \  
/etc/postgresql/16/main/server.crt  
sudo ln -sf /etc/letsencrypt/live/db.example.com/privkey.pem \  
/etc/postgresql/16/main/server.key  
  
sudo chown -h postgres:postgres /etc/postgresql/16/main/server.crt  
sudo chown -h postgres:postgres /etc/postgresql/16/main/server.key  
sudo chmod 600 /etc/postgresql/16/main/server.key
```

PostgreSQL SSL Configuration

```
# /etc/postgresql/16/main/postgresql.conf  
  
#-----  
# SSL/TLS SETTINGS  
#-----  
ssl = on                                # Enable SSL (requires server.crt and server.key)  
  
# Certificate files (paths relative to data directory or absolute)  
ssl_cert_file = 'server.crt'           # Server certificate  
ssl_key_file = 'server.key'            # Server private key (must be readable only by  
postgres)  
ssl_ca_file = 'ca.crt'                 # CA cert for verifying client certificates  
  
# Optional: Certificate Revocation List  
# ssl_crl_file = 'root.crl'  
# ssl_crl_dir = '/etc/postgresql/crl.d'  
  
# TLS Version Control  
ssl_min_protocol_version = 'TLSv1.2'   # Minimum TLS version (default since PG14)  
ssl_max_protocol_version = ''          # Empty = no maximum
```

```
# Cipher Suites (PostgreSQL 14+ syntax using OpenSSL cipher string)
ssl_ciphers = 'HIGH:MEDIUM:+3DES:!aNULL' # Default
# Hardened:
# ssl_ciphers =
'ECDH+AESGCM:DH+AESGCM:ECDH+AES256:!aNULL:!eNULL:!EXPORT:!DES:!RC4:!3DES:!MD5'

# Elliptic curves for ECDH
ssl_ecdh_curve = 'prime256v1'

# DH parameters for ephemeral DH key exchange
# ssl_dh_params_file = 'dh2048.pem' # Generate with: openssl dhparam -out
dh2048.pem 2048

# Performance: prefer server's cipher order
ssl_prefer_server_ciphers = on

# Password (if private key is encrypted) - NOT recommended for automated systems
# ssl_password_command = 'echo secret'
```

Client Certificate Authentication

Setting Up Client Certificates

```
# Create client certificate signed by the same CA

# For user 'appuser':
openssl genrsa -out client.key 2048
openssl req -new -key client.key -out client.csr \
    -subj "/CN=appuser" # CN MUST match the PostgreSQL username

openssl x509 -req -days 365 -in client.csr -CA ca.crt -CAkey ca.key \
    -CAcreateserial -out client.crt

# Protect the key
chmod 600 client.key
```

```
# postgresql.conf: point to CA that signed client certs
ssl_ca_file = 'ca.crt'
```

```
# pg_hba.conf: require certificate authentication
hostssl all appuser 0.0.0.0/0 cert
# 'cert' means: require TLS client cert, CN must match username
```

```
# Client connection with certificate
psql "host=db.example.com \
    dbname=mydb \
    user=appuser \
    sslcert=/path/to/client.crt \
```

```
sslkey=/path/to/client.key \  
sslrootcert=/path/to/ca.crt \  
sslmode=verify-full"
```

pg_ident.conf Certificate Mapping

```
# /etc/postgresql/16/main/pg_ident.conf  
# Map certificate CN to PostgreSQL username  
  
# MAPNAME      CERT-CN          PG-USERNAME  
certmap        john.doe          john_db_user  
certmap        jane.smith        jane_db_user  
certmap        svc-orders@corp.com orders_service  
  
# Allow any user in the 'corp.com' domain  
# (requires regex support – PostgreSQL 16+)  
  
# pg_hba.conf with mapping  
hostssl all all 0.0.0.0/0 cert clientcert=verify-full map=certmap
```

SSL Mode Options

The `sslmode` connection parameter controls what the client does with SSL:

sslmode values (client-side):

disable	- Never use SSL. Connection will be rejected if server requires SSL. Use only in development environments.
allow	- Prefer non-SSL, but use SSL if required by server. Not recommended: insecure default behavior.
prefer	- Try SSL first, fall back to non-SSL if server doesn't support it. DEFAULT in most clients, but not secure enough for production.
require	- Use SSL always. Do NOT verify server certificate. Encrypts traffic but vulnerable to MITM attacks.
verify-ca	- Use SSL, verify that the server certificate is signed by a trusted CA. Protects against MITM if CA is trusted. Does not verify hostname.
verify-full	- Use SSL, verify server certificate AND that CN/SAN matches hostname. MOST SECURE. Use in production. Requires valid server certificate.

Security comparison:

disable	<	allow	<	prefer	<	require	<	verify-ca	<	verify-full
								(no cert	(no hostname	(full
								check)	check)	verification)

For production: always use verify-full

Configuring sslmode in Connection Strings

```
# PSQL
psql "host=db.example.com dbname=mydb user=app sslmode=verify-full
sslrootcert=/path/to/ca.crt"

# Environment variable
export PGSSLMODE=verify-full
export PGSSLROOTCERT=/etc/ssl/pg/ca.crt

# libpq connection string
conn = "host=db.example.com dbname=mydb user=app sslmode=verify-full
sslrootcert=/etc/ssl/pg/ca.crt"
```

```
# Python (psycopg2)
import psycopg2
conn = psycopg2.connect(
    host="db.example.com",
    database="mydb",
    user="appuser",
    password="password",
    sslmode="verify-full",
    sslrootcert="/etc/ssl/pg/ca.crt"
)

# With client certificate:
conn = psycopg2.connect(
    host="db.example.com",
    database="mydb",
    user="appuser",
    sslmode="verify-full",
    sslrootcert="/etc/ssl/pg/ca.crt",
    sslcert="/etc/ssl/pg/client.crt",
    sslkey="/etc/ssl/pg/client.key"
)
```

```
// Java (JDBC)
Properties props = new Properties();
props.setProperty("user", "appuser");
props.setProperty("password", "password");
props.setProperty("ssl", "true");
props.setProperty("sslmode", "verify-full");
props.setProperty("sslrootcert", "/etc/ssl/pg/ca.crt");

Connection conn = DriverManager.getConnection(
    "jdbc:postgresql://db.example.com:5432/mydb", props);
```

Mutual TLS (mTLS)

Mutual TLS requires BOTH server and client to present valid certificates.

mTLS Setup:

1. Server has: server.crt + server.key + ca.crt (CA that signed client certs)
2. Client has: client.crt + client.key + ca.crt (CA that signed server cert)
3. Each verifies the other's certificate

PostgreSQL Configuration for mTLS:

- ssl = on (server has cert)
- ssl_ca_file = ca.crt (trusts client certs signed by this CA)
- pg_hba.conf: hostssl ... cert (requires client cert)
- Client: sslmode=verify-full + sslcert + sslkey + sslrootcert

Certificate Rotation

Zero-Downtime Certificate Rotation

```
# PostgreSQL supports certificate reload without restart (since PG14)

# Step 1: Generate new certificate
openssl req -new -key server.key -out server.csr -subj "/CN=db.example.com"
openssl x509 -req -days 365 -in server.csr -CA ca.crt -CAkey ca.key \
    -CAcreateserial -out server.crt.new

# Step 2: Replace certificate file
cp /etc/postgresql/16/main/server.crt /etc/postgresql/16/main/server.crt.bak
mv server.crt.new /etc/postgresql/16/main/server.crt

# Step 3: Signal PostgreSQL to reload SSL (PostgreSQL 14+)
sudo -u postgres psql -c "SELECT pg_reload_conf();"
# Existing SSL connections continue unaffected
# New connections use the new certificate

# Verify new certificate is loaded
psql -c "SELECT ssl, version, cipher FROM pg_stat_ssl WHERE pid = pg_backend_pid();"

# For key rotation (requires restart):
cp server.key.new /etc/postgresql/16/main/server.key
sudo systemctl reload postgresql # or restart if key changed
```

TLS Configuration Hardening

```
# postgresql.conf – Hardened TLS Configuration

ssl = on
```

```
ssl_cert_file = 'server.crt'
ssl_key_file = 'server.key'
ssl_ca_file = 'ca.crt'

# Force TLS 1.2+
ssl_min_protocol_version = 'TLSv1.2'

# Strong cipher suites only (no weak algorithms)
ssl_ciphers = 'ECDHE-ECDSA-AES256-GCM-SHA384:ECDHE-RSA-AES256-GCM-SHA384:ECDHE-
ECDSA-CHACHA20-POLY1305:ECDHE-RSA-CHACHA20-POLY1305:ECDHE-ECDSA-AES128-GCM-
SHA256:ECDHE-RSA-AES128-GCM-SHA256'

# Server chooses cipher order (not client)
ssl_prefer_server_ciphers = on

# Modern elliptic curves
ssl_ecdh_curve = 'X25519:prime256v1:secp384r1'
```

```
# Test SSL configuration
openssl s_client -connect db.example.com:5432 -starttls postgres
# Check: Protocol version, cipher suite, certificate chain

# Verify minimum TLS version is enforced
openssl s_client -connect db.example.com:5432 -tls1_1 -starttls postgres
# Should fail: no shared cipher / protocol not supported

# Test with specific cipher
openssl s_client -connect db.example.com:5432 -starttls postgres -cipher 'AES256-
SHA'
# Should fail if that cipher is disabled
```

Monitoring SSL Connections

```
-- View SSL status of current connections
SELECT
    pid,
    datname AS database,
    username AS username,
    application_name,
    ssl,
    ssl_version AS tls_version,
    ssl_cipher,
    ssl_bits,
    ssl_client_cert_present,
    ssl_client_dn,
    ssl_issuer_dn
FROM pg_stat_ssl
JOIN pg_stat_activity USING (pid)
WHERE backend_type = 'client backend';
```

```

-- Count SSL vs non-SSL connections
SELECT
    ssl,
    COUNT(*) AS connection_count,
    array_agg(DISTINCT ssl_version) AS versions
FROM pg_stat_ssl
JOIN pg_stat_activity USING (pid)
WHERE backend_type = 'client backend'
GROUP BY ssl;

-- Alert: non-SSL connections from external hosts
SELECT s.pid, a.username, a.client_addr, a.application_name
FROM pg_stat_ssl s
JOIN pg_stat_activity a ON a.pid = s.pid
WHERE s.ssl = false
    AND a.client_addr NOT IN ('127.0.0.1'::inet, '::1'::inet)
    AND a.backend_type = 'client backend';

-- View current server SSL settings
SHOW ssl;
SHOW ssl_cert_file;
SHOW ssl_ciphers;
SHOW ssl_min_protocol_version;

-- Certificate expiration check (from psql)
-- openssl x509 -in server.crt -noout -enddate

```

Common Mistakes

1. **sslmode=prefer or sslmode=require in production** — no certificate verification
2. **Not setting ssl_ca_file** — cannot verify client certificates
3. **Leaving non-SSL host entries when SSL is available** — clients fall back to unencrypted
4. **server.key readable by non-postgres users** — private key compromised
5. **Not adding hostname/IP to SAN** — modern clients reject certificates without SAN
6. **Using outdated TLS 1.0/1.1** — security vulnerabilities
7. **Forgetting certificate expiration monitoring** — surprise outage when cert expires
8. **Not testing client certificate auth** — discovering misconfiguration in production
9. **Using the same cert for all environments** — dev certs should never be trusted in production
10. **Not revoking compromised client certificates** — configure `ssl_crl_file`

Best Practices

1. Always use **verify-full** for production client connections
2. Use **separate CAs** for server and client certificates
3. Set **ssl_min_protocol_version = TLSv1.2**
4. Monitor **certificate expiration** (alert 30 days before)
5. Use **hostssl only** (not `host`) for external connections in `pg_hba.conf`
6. Rotate certificates **before expiration** using `pg_reload_conf` (PG14+)
7. Keep **private keys** with mode 600, owned by postgres

8. Use **SAN (Subject Alternative Names)** not just CN
 9. Use **Let's Encrypt** for publicly-accessible databases with valid hostnames
 10. Test SSL configuration with `openssl s_client` after setup
-

Performance Considerations

SSL overhead:

- Connection establishment: +1-3ms for TLS handshake (one-time per connection)
- Data transfer: ~3-5% CPU overhead for encryption/decryption
- With TLSv1.3 (session resumption): faster reconnection

With connection pooling (PgBouncer):

- TLS terminates at PgBouncer
 - Application → PgBouncer: encrypted (1 TLS handshake per app session)
 - PgBouncer → PostgreSQL: encrypted (1 TLS handshake per pool slot, long-lived)
 - Net result: SSL overhead is amortized across thousands of transactions
-

Interview Questions

Q1: What is the difference between `sslmode=require` and `sslmode=verify-full` ?

A: Both enforce SSL/TLS encryption. `require` just encrypts the connection but does NOT verify the server's certificate. A man-in-the-middle attacker could present a fake certificate and intercept the connection.

`verify-full` verifies that the server certificate is signed by a trusted CA AND that the certificate's CN/SAN matches the hostname. This prevents MITM attacks. Always use `verify-full` in production.

Q2: What files does PostgreSQL need on the server for SSL?

A: (1) `server.crt` — server's certificate (PEM format). (2) `server.key` — server's private key (PEM, mode 600, owned by postgres). (3) `ca.crt` (optional) — CA certificate for verifying client certificates. Enable in `postgresql.conf`: `ssl=on`, `ssl_cert_file`, `ssl_key_file`, `ssl_ca_file`.

Q3: How does certificate authentication work in PostgreSQL?

A: Client presents its TLS client certificate during the handshake. PostgreSQL verifies the certificate is signed by the CA in `ssl_ca_file`. If valid, it extracts the certificate's CN (or SAN) and maps it to a PostgreSQL username (directly or via `pg_ident.conf`). The `pg_hba.conf` must have `hostssl ... cert` for the connection. No password is needed — the private key proves identity.

Q4: Can you reload SSL certificates without restarting PostgreSQL?

A: In PostgreSQL 14+: yes for the server certificate — `pg_reload_conf()` will pick up a new `server.crt`. The server key may still require restart. Existing connections are not affected by reload. In older versions, a full restart is required.

Q5: What is SNI (Server Name Indication) in the context of PostgreSQL SSL?

A: SNI is a TLS extension that allows the client to specify the hostname during the TLS handshake, before the server responds with its certificate. PostgreSQL uses SNI to support multiple SSL certificates on the same server/port when using virtual hosts. Less commonly needed for PostgreSQL, but relevant in environments with multiple databases on different virtual hostnames.

Q6: How do you force all non-local connections to use SSL?

A: In `pg_hba.conf`, replace `host` entries with `hostssl` for all remote connections. Remove any `host` entries for remote addresses (or add `host ... reject` for non-SSL). Also remove `hostnoss1` entries. Clients using `sslmode=disable` or `sslmode=allow` will be rejected.

Q7: What is a Certificate Revocation List (CRL) and how do you use it in PostgreSQL?

A: A CRL is a list of certificates that have been revoked before their expiration date (e.g., a compromised client key). PostgreSQL can check client certificates against a CRL: set `ssl_crl_file = 'root.crl'` or `ssl_crl_dir` in `postgresql.conf`. If the client presents a certificate listed in the CRL, authentication is rejected.

Q8: How do you test that PostgreSQL SSL is correctly configured?

A: (1) `openssl s_client -connect host:5432 -starttls postgres` — verify TLS version and cipher. (2) Connect with `psql ... sslmode=verify-full` — if it connects, certificate is valid. (3) `SELECT ssl, ssl_version, ssl_cipher FROM pg_stat_ssl WHERE pid=pg_backend_pid()` — verify from inside. (4) Try connecting with `sslmode=disable` to a `hostssl` entry — should fail.

Exercises and Solutions

Exercise 1: Complete SSL Setup Script

```
#!/bin/bash
# setup_postgresql_ssl.sh
PGDATA=/var/lib/postgresql/16/main
HOSTNAME=db.example.com
DAYS_VALID=365

# Generate CA
openssl genrsa -out $PGDATA/ca.key 4096
openssl req -new -x509 -days 3650 -key $PGDATA/ca.key -out $PGDATA/ca.crt \
    -subj "/CN=PostgreSQL Internal CA"

# Generate server cert with SAN
cat > /tmp/san.conf << EOF
[req]
distinguished_name = req_distinguished_name
req_extensions = v3_req
[req_distinguished_name]
[v3_req]
subjectAltName = DNS:${HOSTNAME},IP:$(hostname -I | awk '{print $1}')
EOF

openssl genrsa -out $PGDATA/server.key 2048
openssl req -new -key $PGDATA/server.key -out /tmp/server.csr \
    -subj "/CN=${HOSTNAME}" -config /tmp/san.conf
openssl x509 -req -days $DAYS_VALID -in /tmp/server.csr \
    -CA $PGDATA/ca.crt -CAkey $PGDATA/ca.key -CAcreateserial \
    -out $PGDATA/server.crt -extfile /tmp/san.conf -extensions v3_req
```

```
# Set permissions
chmod 600 $PGDATA/server.key $PGDATA/ca.key
chmod 644 $PGDATA/server.crt $PGDATA/ca.crt
chown postgres:postgres $PGDATA/server.* $PGDATA/ca.*

# Update postgresql.conf
cat >> $PGDATA/postgresql.conf << EOF
ssl = on
ssl_cert_file = 'server.crt'
ssl_key_file = 'server.key'
ssl_ca_file = 'ca.crt'
ssl_min_protocol_version = 'TLSv1.2'
ssl_prefer_server_ciphers = on
EOF

echo "SSL setup complete. Reload PostgreSQL to apply."
```

Cross-References

- [01 authentication methods.md](#) — Certificate auth in pg_hba.conf
- [07 secrets management.md](#) — Certificate lifecycle management
- [../13 Replication HA/02 streaming_replication.md](#) — SSL for replication connections